Team 12: Automated Bear Detection and Deterrence

Nathan Auten, Jacob Bidorini, Justin Latif, Bryan Martinez
Sponsor: Interface Technologies
Sponsor Advisor: Holly Beum
Faculty Advisor: Yuan Hong

Background

Within the state of Connecticut, there has been a rise in the number of incidents in which black bears have entered people's homes. These incidents have often led to injury of people's pets as well as damage to their property.

The goal of this project is to create a device that homeowners can deploy around their property that will detect the presence of a bear and, using overwhelming stimulus, scare the bear away.

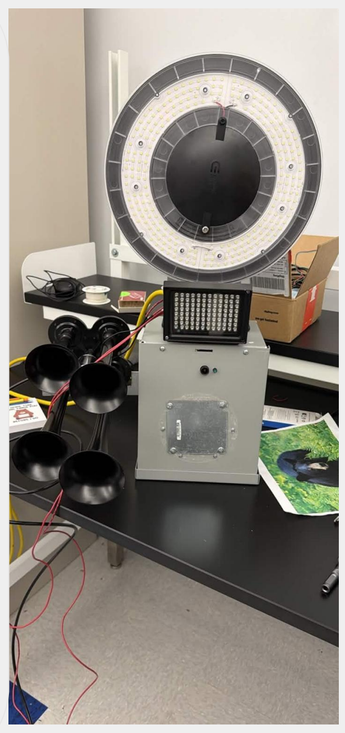
Previous Year vs. This Year

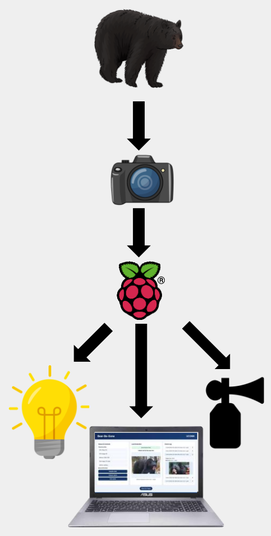
Last years team was able to develop a functioning prototype, but in field testing was unable to deter a bear.

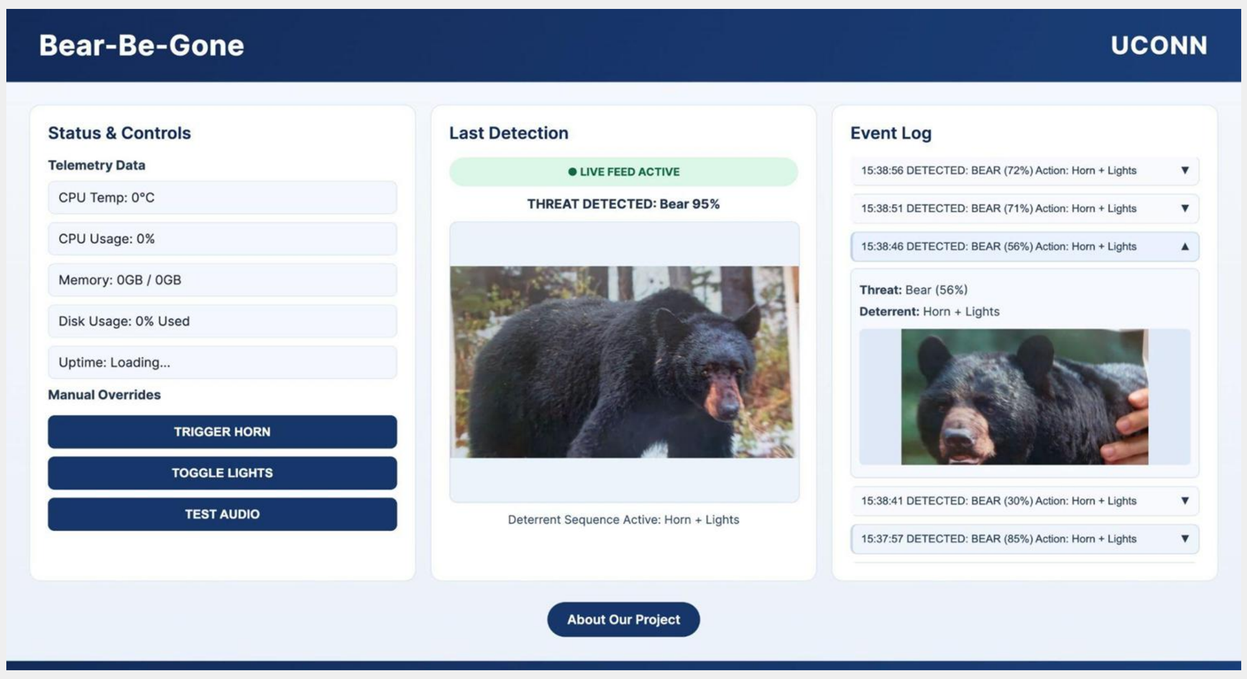
In order to improve this year, we have increased both the amount of light and sound generated by the device. We also wanted to improve the efficiency of the device through model optimization and a clean, simple frontend for users.

Device Overview

- Raspberry Pi 5
- Generic USB camera
- NCNN Compiled YOLOv11 Computer Vision model
- 120 decibel truck horn
- 12000 lumen led light
- Flask webserver
- Vite React app frontend





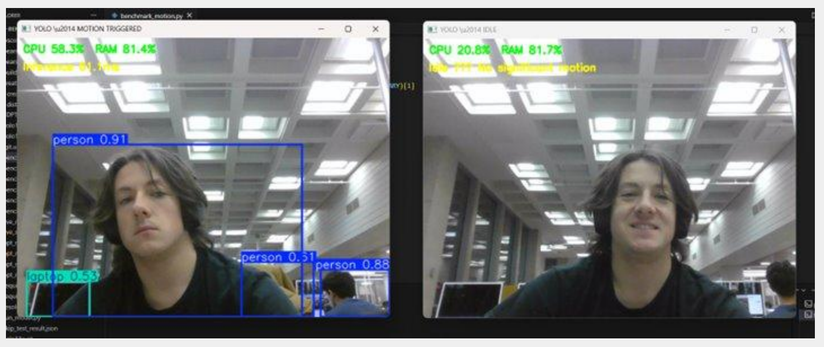


Frontend & Backend

- Frontend:
  - A streamlined design with various interactable features and important info
  - Includes Pi Statistics, Latest Capture, Event Logs, and Manual Overrides
  - Everything needed on a single page to interface with hardware optionally
- Backend:
  - Asynchronous Multi-Threaded Pipeline: Decouple primary detection loop with network tasks to prevent network latency from stalling deterrent loop
  - Push-Based Communication: Utilize WebSockets allowing server to push detection data, images, and telemetry to dashboard instantly
  - Server-Sent Events (SSE): Allow users to remotely trigger airhorn or strobe lights from dashboard without needing additional network configurations
  - Health Telemetry: Periodically monitor and transmit system metrics of the Pi using psutil

Model Improvements

- Tested a variety of optimization techniques to improve the YOLO model's performance on the Raspberry Pi
- Resolution Downscaling
  - Lowered the resolution of the image being processed by the model, reducing number of pixels in calculation
- Motion Detection
  - Model only runs when there is a large difference between the current and previous frames



- NCNN Model Format
  - Converted the model from Pytorch to NCNN, a format optimized for usage on ARM processors

```
==================== RESULTS ====================
Model          | Latency  | FPS    | CPUavg  | CPUpeak |
-------------------------------------------------------
11n_PT32       | 431.6    | 2.3    | 96.2    | 100.0   |
11n_NCNN_FP32  | 93.3     | 10.7   | 92.3    | 100.0   |
```

Performance comparison of .pt vs NCNN model format



Live bear captured in field testing

Field Testing & Results

Once we received approval from the IACUC, we were able to deploy the device in our sponsors backyard for field testing. The device, alongside game cameras to capture photos and videos, was deployed and aimed toward two towels that were sprayed with bear attractant. Within 3 days of deployment, a bear approached the device, and it triggered, causing the bear to run away.

Overall, we observed that our increased stimuli was key in the device's ability to deter bears. Paired with NCNN compiled YOLOv11n model, motion detection, and resolution downscaling, the device was able to run at much higher efficiency, with the device idling around ~0% CPU usage when no motion is detected. Alongside the streamlined frontend, there is now an effective end-to-end pipeline of detection, deterrence, and ability for the user to see the logs of events and monitor the health of the device.



Test site